EAST WEST UNIVERSITY

Department of Computer Science and Engineering (CSE)
Semester: Spring 2026, B.Sc. in CSE

BrandON: A Web-Based Banner Order Management Platform

Software Engineering Project Report
CSE412

Submitted to:
Dr. Mohammad Mahdi Hassan
Associate Professor
Department of Computer Science & Engineering

Submitted by: Group-13 (Team Trelet)

Mahir Faysal Haque Dipto	2022-2-60-044
Sonia Binte Siraj	2022-2-60-087
Sumaya Binte Naser	2022-2-60-155
Raima Husna	2022-2-60-052
Mubashir Jawad	2022-2-60-039

April 2026

0 Contents

1 Introduction **4**
1.1 Problem Statement . . . 4
1.2 Proposed Solution . . . 4
1.3 Project Scope . . . 4
1.4 Technology Choices . . . 5

2 User Stories and Requirements **5**
2.1 Customer Stories . . . 5
2.2 Admin Stories . . . 5
2.3 Acceptance Criteria (Selected) . . . 6

3 System Architecture and Design **6**
3.1 System Architecture Overview . . . 6
3.2 Database Design . . . 9
3.2.1 Relationships . . . 9
3.3 Sequence Diagrams . . . 10
3.3.1 Place Order Flow . . . 10
3.3.2 Admin Login Flow . . . 11
3.4 Use Case Diagram . . . 12

4 Design Patterns **12**
4.1 Singleton Pattern: Database Client . . . 12
4.1.1 The Problem It Solves . . . 12
4.1.2 Implementation . . . 12
4.1.3 Key Design Elements . . . 13
4.2 Strategy Pattern: Pricing Calculator . . . 13
4.2.1 The Problem It Solves . . . 13
4.2.2 Key Design Elements . . . 15

5 UI and GUI Design **15**
5.1 Color Palette . . . 15
5.2 Typography . . . 15
5.3 Navigation Structure . . . 16

6 Sprint Scheduling and Project Management **16**
6.1 Team Development Strategy . . . 16
6.2 Sprint 6 Detailed Tasks . . . 17
6.3 Team Contribution Summary . . . 18

7 Software Estimation **18**
7.1 KLOC Estimation . . . 18
7.2 Function Point Analysis . . . 19
7.3 Effort Estimation (COCOMO Basic) . . . 19
7.4 Duration Estimation . . . 20
7.5 Team Size . . . 20

8 Software Testing **20**

8.1 Testing Strategy 20
8.2 Test Architecture 21
8.3 Unit Testing 21
8.3.1 Validation Module (17 tests) 21
8.3.2 Pricing Strategy Module (16 tests) 21
8.4 Integration Testing 22
8.4.1 Materials API (15 tests) 22
8.4.2 Orders API (21 tests) 22
8.4.3 Gallery API (9 tests) 22
8.4.4 Contact & Auth API (8 tests) 22
8.5 System Testing 22
8.6 Regression Testing 23

9 Conclusion **23**

Appendix **24**

1 Introduction

1.1 Problem Statement

Local banner printing shops operate entirely offline. A customer who needs a banner must visit the shop in person, wait while pricing is looked up, negotiate manually, and return at a later date to collect the order. There is no online order tracking, no transparent pricing, and no formal record of transactions. This creates inconvenience for customers and restricts the shop's ability to reach a wider audience.

1.2 Proposed Solution

BrandON is a web-based platform that digitalises the banner ordering workflow. Customers can browse available materials with transparent per-square-foot pricing, configure custom dimensions, upload their design file, and place an order without visiting the shop or creating an account.

The administration side provides a password-protected dashboard where the shop owner can manage materials, process orders through a defined status pipeline (Pending → Confirmed → In Progress → Completed), upload portfolio work to the gallery, and keep company information up to date.

- Browsing active banner materials with price-per-square-foot
- Custom dimension input (width × height in feet) with automatic total price calculation
- Design file upload (image/PDF) per order
- Order placement without requiring customer account registration
- Admin dashboard for full CRUD management of materials, orders, and gallery
- Order status pipeline enforcement with history tracking
- Portfolio gallery management for past work

1.3 Project Scope

The platform is designed for a single banner printing shop. Multi-store support is outside the project scope. Payment processing is not included, as the shop operates on a Cash on Delivery model. Customer registration is not required; orders are placed using only a name and phone number.

1.4 Technology Choices

Table 1: Technology Stack

Layer	Technology	Reason
Frontend	Next.js 14 (React)	Server-side rendering, App Router, file-based routing
Language	TypeScript	Type safety catches errors before runtime
Styling	Tailwind CSS	Utility-first; no separate stylesheet files
ORM	Prisma 5	Type-safe queries, schema-as-code, migration tracking
Database	PostgreSQL 15	Reliable relational DB, runs in Docker
Testing	Jest 30 + ts-jest	Industry-standard; supports TypeScript natively
Deployment	Docker Compose	Single command to start the database container

2 User Stories and Requirements

2.1 Customer Stories

ID	As a...	I want to...	So that...
US-1	Customer	Browse available banner materials with prices	I can compare options before ordering
US-2	Customer	See the price per square foot for each material	I know exactly what I will pay
US-3	Customer	Enter custom width and height for my banner	I get the exact size I need
US-4	Customer	See the total price calculated automatically	I don't have to do mental arithmetic
US-5	Customer	Upload my design file (image or PDF)	The shop can print my custom design
US-6	Customer	Place an order with just my name and phone	I don't need to create an account
US-7	Customer	View the shop's portfolio of past work	I can judge quality before committing
US-8	Customer	See contact information and location	I can reach the shop directly if needed

2.2 Admin Stories

ID	As a...	I want to...	So that...
US-9	Admin	Add materials with name, price, and description	Customers see what's available
US-10	Admin	Edit or deactivate existing materials	I can update prices or remove discontinued items
US-11	Admin	View all incoming orders sorted by date	I don't miss any orders
US-12	Admin	Update order status through the pipeline	Orders progress forward predictably
US-13	Admin	Upload images to the portfolio gallery	Potential customers can see past work
US-14	Admin	Update company profile and contact details	Public pages always show correct information
US-15	Admin	Access the panel with a secret password	Only authorized staff can manage the shop

2.3 Acceptance Criteria (Selected)

US-3: Custom Dimensions

Given the customer is on the order form and enters width = 4 ft, height = 3 ft, material = Vinyl ($15/sqft), quantity = 2, *then* the total price displayed is $4 \times 3 \times 15 \times 2 = \$360$. , **US-6: Place Order Without Account**

Given a customer fills in name, phone, material, and dimensions, *when* they submit the form, *then* an order is saved with `status = PENDING` and a con, *when* they submit the form, *then* an order is saved with `status = PENDING` and a confirmation message display

Given a user navigates to `/admin` with no valid cookie, *then* they are redirected to `/admin/login`. After entering the correct admin password, a session cookie is set and access to the dashboard is granted. , *then* they are redirected to `/admin/login`. After entering the correct admin password, a session cookie is set and access to the dashboard

3 System Architecture and Design

3.1 System Architecture Overview

The system is divided into four clear layers: the browser client, the Next.js server (App Router), the library modules, and the database. Figure 1 illustrates the full component flow.

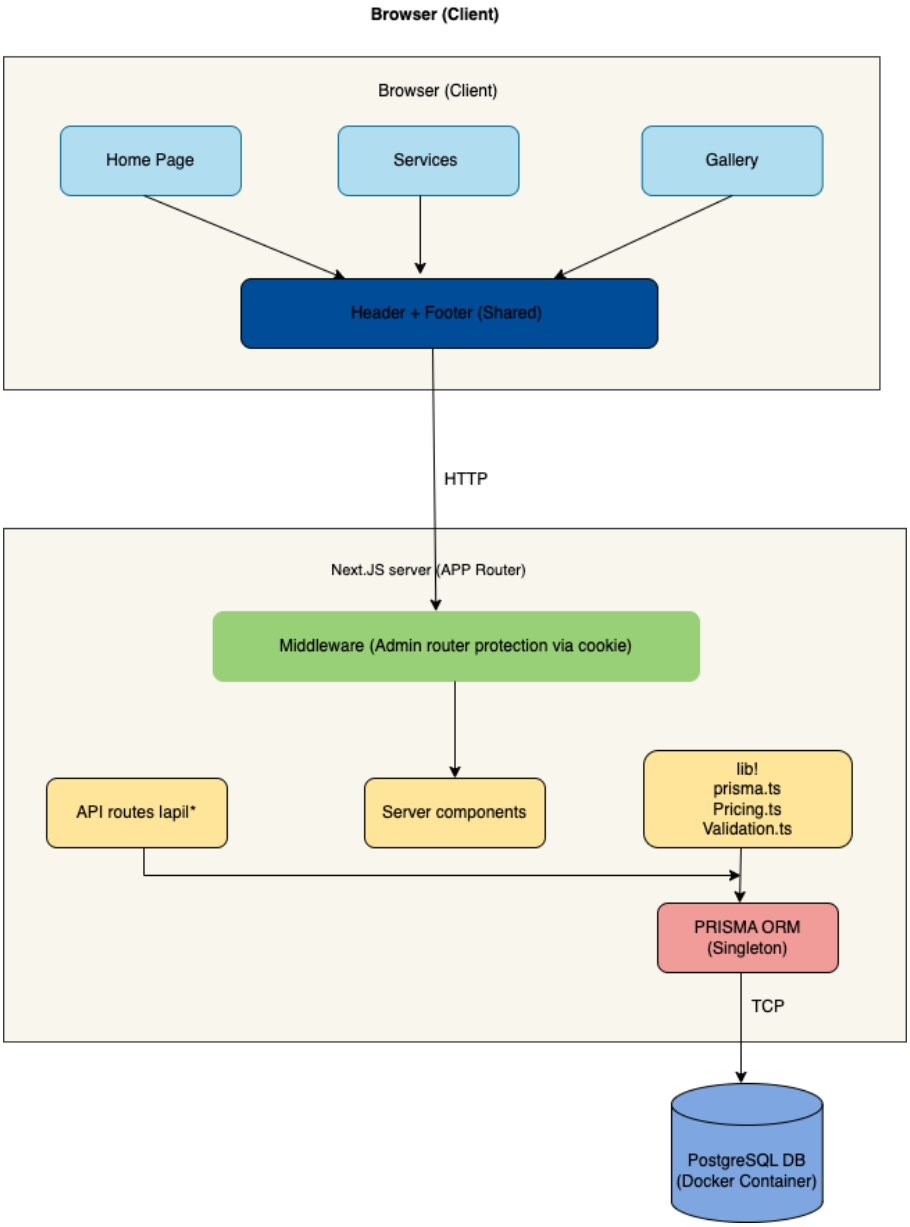

Figure 1: System Architecture Diagram. The browser communicates with the Next.js server over HTTP. The middleware layer intercepts all `/admin/*` routes and enforces cookie-based authentication before any server component or API route is reached. The `lib/` modules (prisma.ts, pricing.ts, validation.ts) are shared utilities consumed by API routes and server components alike. All database operations go through Prisma ORM to PostgreSQL running in a Docker container.

Every incoming HTTP request first passes through the middleware layer. For public routes, the request is forwarded directly to the appropriate server component. For any `/admin/*` path, the middleware verifies the `admin_token` cookie against the `ADMIN_SECRET` environment variable. Unauthenticated requests are redirected to the login page. No external authentication provider or JWT library is used; the implementation relies on a single cookie checked in the middleware file.

Every incoming HTTP request first passes through the middleware layer. For public

routes, the request is forwarded directly to the appropriate server component. For any `/admin/*` path, the middleware verifies the `admin_token` cookie against the `ADMIN_SECRET` environment variable. Unauthenticated requests are redirected to the login page. No external authentication provider or JWT library is used; the implementation relies on a single cookie checked in the middleware fil

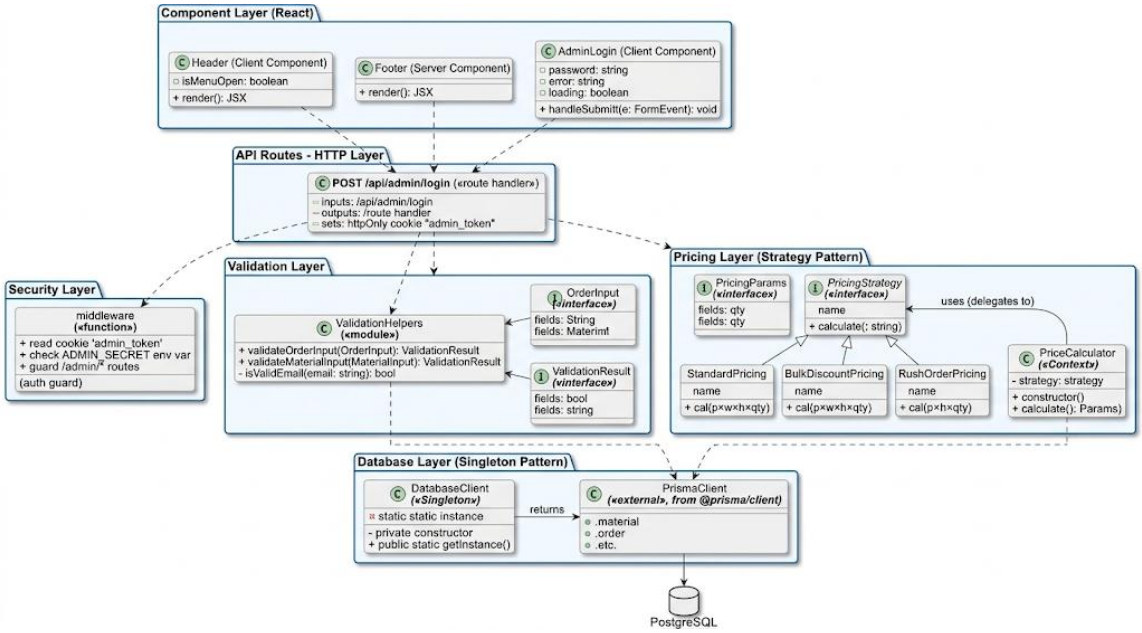

Figure 2: Class and Component Diagram. The diagram shows the five distinct layers of the system: the React Component Layer (Header, Footer, AdminLogin), the API Routes HTTP Layer (POST /api/admin/login), the Validation Layer (ValidationHelpers module with OrderInput and MaterialInput interfaces), the Pricing Layer implementing the Strategy Pattern (PricingStrategy interface, three concrete strategies, and PriceCalculator context), and the Database Layer using the Singleton Pattern (DatabaseClient wrapping PrismaClient). Arrows indicate delegation and composition relationships between layers.

3.2 Database Design

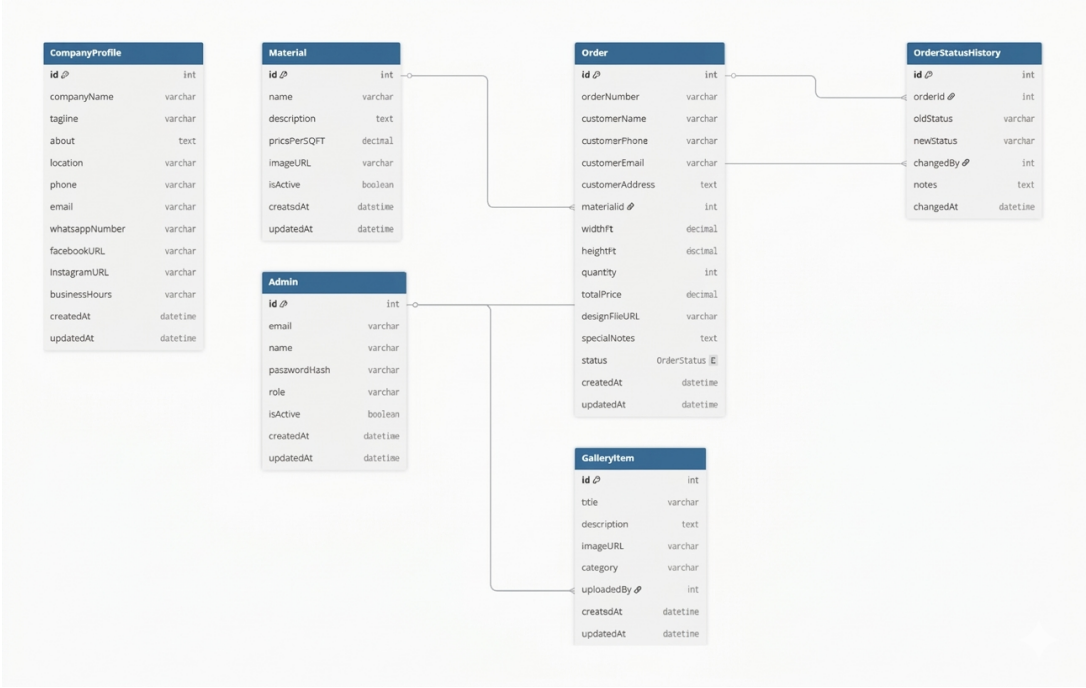

Figure 3: Entity Relationship Diagram. Six tables constitute the schema: `Material` (products with pricing), `Order` (customer submissions with foreign key to Material), `OrderStatusHistory` (audit log of every status change), `Admin` (shop staff accounts with bcrypt-hashed passwords), `GalleryItem` (portfolio images uploaded by admins), and `CompanyProfile` (a single-row settings table for shop contact details and social links). The diagram shows all primary keys, foreign key relationships, and field types.

3.2.1 Relationships

Table 4: Database Relationship Summary

From	To	Description
Material	Order	One material can appear in many orders (1:M)
Order	OrderStatusHistory	One order accumulates many status change records (1:M)
Admin	OrderStatusHistory	One admin can change status on many orders (1:M)
Admin	GalleryItem	One admin can upload many gallery images (1:M)
CompanyProfile	N/A	Single-row configuration table; no foreign key relationships

3.3 Sequence Diagrams

3.3.1 Place Order Flow

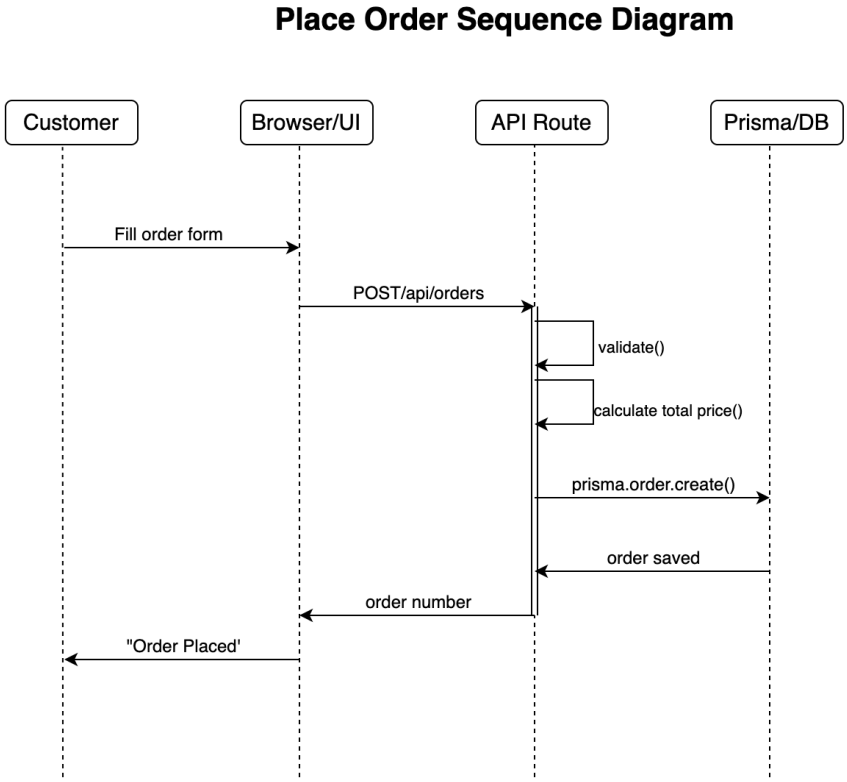

Figure 4: Place Order Sequence Diagram. The sequence begins when the customer fills the order form and submits it. The browser sends a POST request to `/api/orders`. The API route first calls `validateOrderInput()`; any validation failure returns a 400 response immediately. For valid input, the server fetches the material price from the database and recalculates the total price on the server side, ignoring any value submitted by the client. Prisma creates the order record, generates a human-readable order number in the format BON-YYYYMMDD-NNN, and the confirmation is returned to the customer.

3.3.2 Admin Login Flow

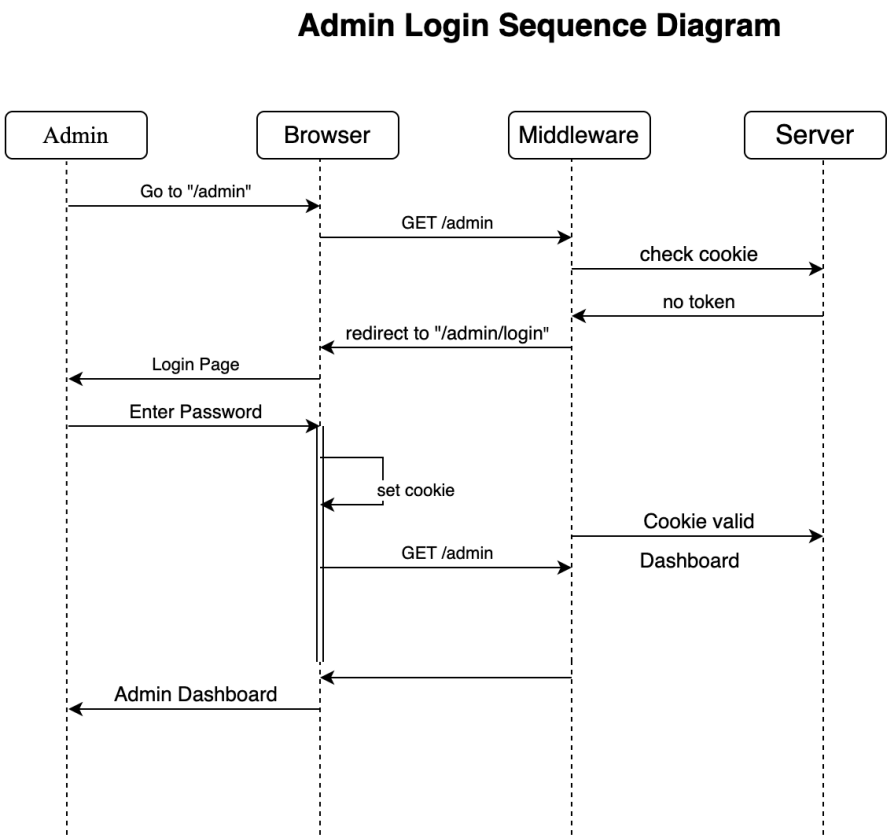

Figure 5: Admin Login Sequence Diagram. When the admin navigates to `/admin`, the middleware reads the `admin_token` cookie. If the cookie is absent, the request is redirected to `/admin/login`. The admin submits their password via a POST request to `/api/admin/login`. The server compares the submitted value against the `ADMIN_SECRET` environment variable. On success, an httpOnly cookie named `admin_token` is set and the admin is redirected to the dashboard. All subsequent requests to `/admin/*` routes repeat this cookie check through the middleware; the dashboard pages themselves perform no additional authentication logic.

3.4 Use Case Diagram

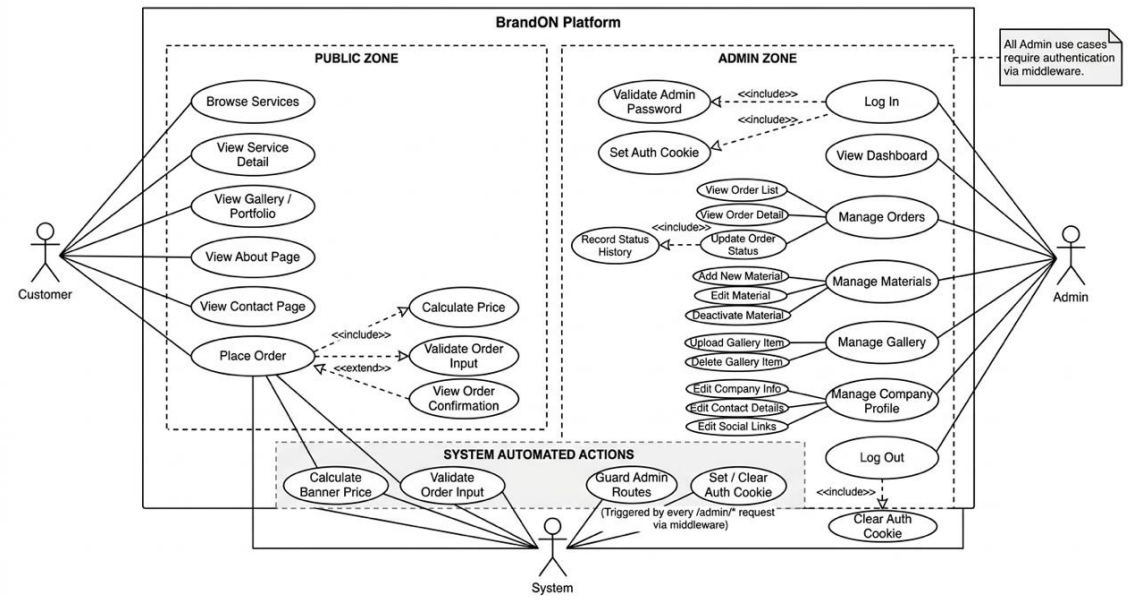

Figure 6: Use Case Diagram for the BrandON Platform. The diagram is partitioned into three zones: Public Zone (customer-facing actions), Admin Zone (authenticated management actions), and System Automated Actions (middleware-triggered operations). Customer use cases include browsing materials, viewing gallery, placing orders, and contacting the shop. Admin use cases cover the full CRUD lifecycle for materials, orders, and gallery items, plus company profile management. The "Place Order" use case includes "Validate Order Input" and "Calculate Price" as included sub-flows. All Admin Zone use cases include authentication via middleware as a prerequisite, shown with «include» relationships.

4 Design Patterns

Two Gang-of-Four design patterns were applied in this project. Each pattern addresses a specific technical problem that would otherwise lead to errors in a production environment.

4.1 Singleton Pattern: Database Client

File: `lib/prisma.ts`

4.1.1 The Problem It Solves

Next.js development mode uses hot-module replacement, which causes the server to partially restart whenever a file is saved. Without the Singleton pattern, each restart would create a new `PrismaClient` instance. PostgreSQL has a default connection limit of 100. Without this pattern, repeated hot reloads would exhaust the connection pool and cause the application to fail with a "too many connections" error during development.

4.1.2 Implementation

```
class DatabaseClient {
  private static instance: PrismaClient | undefined
  private constructor() {}

  public static getInstance(): PrismaClient {
    if (!DatabaseClient.instance) {
      DatabaseClient.instance = new PrismaClient({
        log: process.env.NODE_ENV === 'development' ? ['query'] : []
      })
    }
    return DatabaseClient.instance
  }
}

// Survive Next.js hot reloads by anchoring to globalThis
const globalForPrisma = globalThis as unknown as {
  prisma: PrismaClient | undefined
}
const prisma = globalForPrisma.prisma ?? DatabaseClient.getInstance()
if (process.env.NODE_ENV !== 'production') globalForPrisma.prisma = prisma

export default prisma
```

Listing 1: Singleton Pattern in lib/prisma.ts (TypeScript)

4.1.3 Key Design Elements

Table 5: Singleton Pattern Elements

Element	Purpose
`private constructor()`	Blocks `new DatabaseClient()` from any external code
`private static instance`	Holds the single shared PrismaClient reference
`getInstance()`	Creates on first call; returns the same instance every subsequent call
`globalForPrisma`	Preserves the instance across Next.js hot reloads in development

4.2 Strategy Pattern: Pricing Calculator

File: `lib/pricing.ts`

4.2.1 The Problem It Solves

The BrandON platform supports three pricing modes. Standard orders apply a base area calculation. Bulk orders receive a 10% discount when the quantity reaches five or

more. Rush orders carry a 25% surcharge. Without the Strategy pattern, all three rules would be placed in a single function with multiple conditional branches, and adding a new pricing rule would require modifying that function and retesting all existing behaviour.

The Strategy pattern isolates each pricing rule in its own class. New rules can be introduced by adding a new class without modifying any existing code.

```
export interface PricingStrategy {
  name: string
  calculate(params: PricingParams): number
}

// Standard: base area x quantity
export class StandardPricing implements PricingStrategy {
  name = 'Standard'
  calculate({ pricePerSqFt, widthFt, heightFt, quantity }: PricingParams) {
    return Math.round(pricePerSqFt * widthFt * heightFt * quantity * 100) / 100
  }
}

// Bulk: 10% off for 5+ units
export class BulkDiscountPricing implements PricingStrategy {
  name = 'Bulk Discount (10% off for 5+)'
  calculate({ pricePerSqFt, widthFt, heightFt, quantity }: PricingParams) {
    const base = pricePerSqFt * widthFt * heightFt * quantity
    const discount = quantity >= 5 ? 0.10 : 0
    return Math.round(base * (1 - discount) * 100) / 100
  }
}

// Rush: +25% surcharge
export class RushOrderPricing implements PricingStrategy {
  name = 'Rush Order (+25%)'
  calculate({ pricePerSqFt, widthFt, heightFt, quantity }: PricingParams) {
    return Math.round(pricePerSqFt * widthFt * heightFt * quantity * 1.25 * 100) / 100
  }
}

// Context: delegates to whichever strategy is active
export class PriceCalculator {
  private strategy: PricingStrategy
  constructor(strategy: PricingStrategy = new StandardPricing()) {
    this.strategy = strategy
  }
  setStrategy(strategy: PricingStrategy): void { this.strategy = strategy }
  calculate(params: PricingParams): number { return this.strategy.
```

```
    calculate(params) }
}
```

Listing 2: Strategy Pattern in lib/pricing.ts (TypeScript)

4.2.2 Key Design Elements

Table 6: Strategy Pattern Elements

Element	Purpose
`PricingStrategy` interface	Defines the `calculate()` contract all strategies must fulfil
`StandardPricing`	Base calculation: price × area × quantity
`BulkDiscountPricing`	10% discount when quantity ≥ 5
`RushOrderPricing`	25% surcharge added to base price
`PriceCalculator`	Context class; holds a strategy and delegates calculation
`setStrategy()`	Allows runtime switching without reconstructing the calculator

This follows the Open/Closed Principle: the existing strategies are closed for modification, and the system is open for extension by adding new strategy classes.

5 UI and GUI Design

5.1 Color Palette

Table 7: Brand Color Palette

Name	Hex	Usage
Primary	`#22C55E`	Green: CTA buttons, highlights, active links
Secondary	`#1E293B`	Dark slate: card backgrounds, nav bar
Accent	`#3B4B63`	Blue-grey: icon backgrounds, borders
Base	`#0F172A`	Very dark navy: page background
Text Primary	`#FFFFFF`	White: all headings and body text
Text Muted	`#9CA3AF`	Grey: descriptions and captions

5.2 Typography

The project uses Tailwind's default `font-sans` system font stack. Headings are bold and white; body text is displayed in gray-300 to gray-400. The hero section uses responsive text scaling (`text-5xl` on mobile, `text-6xl` on medium screens, `text-7xl` on large screens), managed through Tailwind breakpoint utility classes without any custom CSS.

- **Mobile:** Single-column layout. The Header component uses a `useState` boolean to toggle the hamburger menu.
- **Tablet (md: 768px):** Two-column card grids for material listings and gallery.
- **Desktop (lg: 1024px):** Three to four column layouts, full horizontal navigation.

5.3 Navigation Structure

Table 8: Public Navigation Routes

Label	Route	Description
Home	`/`	Landing page with hero and feature overview
Services	`/services`	Banner materials catalogue with pricing
Portfolio	`/gallery`	Past work showcase
About	`/about`	Company information
Contact	`/contact`	Location, phone, email
Admin	`/admin`	Protected dashboard (not in public nav)

6 Sprint Scheduling and Project Management

6.1 Team Development Strategy

The project followed a two-phase development approach. During the first four sprints, Mahir led the core technical work, including system architecture, database schema design, Docker configuration, and admin middleware. During this same period, the remaining team members focused on setting up their local development environments, running the application, and familiarising themselves with React component structure, Prisma queries, and the Next.js directory layout through hands-on exploration.

This preparation phase allowed the team to contribute effectively from Sprint 5 onwards. From that point, work was distributed by functional area: Jawad was responsible for the backend API layer, Sonia and Saina managed the admin CRUD interfaces and database migrations, Raima developed the customer-facing order flow, and Sumaya handled layout, responsive design, and UI polish

Table 9: Sprint Summary Table

Sprint	Phase	Key Deliverables
1	Discovery	Client meetings, PRD finalization, brand style guide acquisition
2	Infrastructure	Docker Compose setup, Next.js scaffold, Tailwind + ESLint configuration
3	Architecture	Prisma schema (Material, Order, Admin, GalleryItem, CompanyProfile), Singleton pattern
4	Security	Admin cookie-based middleware, mock data for UI, RBAC route isolation
5	Transition	Public page shells, client demo, first distributed commits from team
6	Core Backend	All API routes (materials, orders, gallery), design patterns, 33 unit tests
7	Order Flow	Customer order page (4-step), admin CRUD tables, order number generation
8	Hardening	Server/client validation, file upload guards, loading and error states
9	Polish & QA	Responsive pass, regression testing, documentation, final demo preparation

6.2 Sprint 6 Detailed Tasks

#	Task	Assigned	Status
1	Next.js 14, TypeScript, Tailwind, Prisma project setup	Mahir	Done
2	Docker Compose for PostgreSQL	Mahir	Done
3	Prisma schema: all 5 models	Jawad	Done
4	Admin middleware cookie-based auth	Mahir	Done
5	Admin dashboard page	Raima	Done
6	Public layout: Header and Footer components	Sumaya	Done
7	Home page: hero, how-it-works, why-choose sections	Sumaya	Done
8	Services page with material cards	Raima	Done
9	Gallery page with admin management	Saina	Done
10	About & Contact pages	Sumaya	Done
11	Singleton pattern: prisma.ts	Mahir	Done
12	Strategy pattern: pricing.ts	Mahir	Done
13	Input validation helpers: validation.ts	Jawad	Done
14	Unit tests: pricing and validation (33 tests)	Mahir	Done

Continued

#	Task	Assigned	Status
15	Jest + ts-jest multi-project configuration	Mahir	Done

6.3 Team Contribution Summary

Table 11: Member Roles and Verified Contributions

Member	Role	Primary Contributions
Mahir	Full-Stack Lead	Architecture, auth middleware, design patterns, testing infrastructure, final integration and QA
Jawad	Backend	All API routes (materials, orders, gallery, upload, contact), Prisma queries, order number generation
Raima	Frontend/React	Customer order page (4-step flow), services page, client-side validation and feedback UX
Sonia/Saina	Frontend/DB	Admin CRUD UIs for materials and orders, database migrations, seed scripts, gallery management
Sumaya	UI/Responsive	Header, Footer, Home page, About, Contact pages, layout polish, responsive design pass

7 Software Estimation

7.1 KLOC Estimation

Lines of code were counted across all project source files (`.ts` and `.tsx`) excluding configuration files, generated Prisma client code, and test files.

Table 12: KLOC Breakdown by Module

Module	Description	Estimated LOC
`app/(public)/`	Public pages (Home, Services, Gallery, About, Contact, Order)	820
`app/(admin)/`	Admin pages (Dashboard, Materials, Orders, Gallery, Messages)	730
`app/api/`	All API route handlers (8 endpoints)	650
`lib/`	pricing.ts, validation.ts, prisma.ts, mockServices.ts	280
`components/`	Header, Footer, LoadingSpinner, MaterialCard, OrderFormModal	420
`middleware.ts`	Admin route protection logic	40
`prisma/schema`	Database schema definition	80
Total		**≈ 3,020**

Total estimated source size: **3.02 KLOC** (approximately 3,000 non-blank, non-comment lines of TypeScript/TSX).

7.2 Function Point Analysis

Function Points were estimated using the IFPUG method. Five component types are counted: External Inputs (EI), External Outputs (EO), External Inquiries (EQ), Internal Logical Files (ILF), and External Interface Files (EIF).

Table 13: Function Point Count

Type	Component	Complexity	Count
EI	Place order form submission	Average (4)	1
EI	Admin login	Low (3)	1
EI	Add/Edit material (admin)	Average (4)	2
EI	Update order status	Low (3)	1
EI	Upload gallery image	High (6)	1
EI	Submit contact form	Low (3)	1
EO	Order confirmation with number	Average (5)	1
EO	Price calculation display	Low (4)	1
EQ	List materials (public)	Low (3)	1
EQ	List orders (admin)	Average (4)	1
EQ	List gallery items (public)	Low (3)	1
EQ	Get single order detail	Low (3)	1
ILF	Material, Order, GalleryItem, Admin, OrderStatusHistory, CompanyProfile	Average (10)	6
Unadjusted FP (UFP)			

UFP = (3+3+4+4+3+3+4)·EI weights+(5+4)·EO weights+(3+4+3+3)·EQ weights+6×10

$$\text{UFP} = 24 + 9 + 13 + 60 = \textbf{106 Function Points}$$

Applying a Value Adjustment Factor (VAF) of 1.0 (moderate system complexity, no distributed processing, no performance constraints beyond basic web app norms):

$$\text{AFP} = \text{UFP} \times \text{VAF} = 106 \times 1.0 = \textbf{106 AFP}$$

7.3 Effort Estimation (COCOMO Basic)

Using the Basic COCOMO model for an *organic*-type project (small team, familiar environment, well-understood requirements):

$$E = a \cdot (\text{KLOC})^b$$

For organic mode: $a = 2.4$, $b = 1.05$

$$E = 2.4 \times (3.02)^{1.05} \approx 2.4 \times 3.22 \approx \mathbf{7.73} \quad \textbf{person-months}$$

7.4 Duration Estimation

$$D = c \cdot E^d$$

For organic mode: $c = 2.5$, $d = 0.38$

$$D = 2.5 \times (7.73)^{0.38} \approx 2.5 \times 2.25 \approx \mathbf{5.6} \quad \textbf{months}$$

7.5 Team Size

$$N = \frac{E}{D} = \frac{7.73}{5.6} \approx \mathbf{1.4} \quad \textbf{people}$$

The COCOMO model suggests approximately 1 to 2 developers for a project of this size. This is broadly consistent with the actual team structure, where one primary lead managed the bulk of the core development work and four contributors supported specific functional areas across a 9-sprint cycle.

Table 14: Estimation Summary

Metric	**Value**
Total KLOC	3.02 KLOC
Function Points	106 AFP
Effort (COCOMO)	7.73 person-months
Duration (COCOMO)	5.6 months
Recommended Team	1–2 developers
Actual Team	5 members over 9 sprints

8 Software Testing

8.1 Testing Strategy

The test suite is organised as a Jest multi-project configuration, where each test level runs in its own isolated project with separate setup, teardown, and timeout settings. Unit tests have no external dependencies and complete in under one second without requiring Docker. Integration, system, and regression tests require a running Next.js development server and a live PostgreSQL database.

The four-level structure provides progressively broader coverage. Unit tests validate isolated business logic. Integration tests confirm HTTP API contracts. System tests verify end-to-end user journeys. Regression tests guard against the reintroduction of previously fixed bugs.

Table 15: Test Results Summary

Level	Suites	Tests	Passed	Failed	Pass Rate
Unit	2	33	33	0	100%
Integration	4	53	53	0	100%
System	1	24	24	0	100%
Regression	1	17	17	0	100%
Total	**8**	**133**	**133**	**0**	**100%**

Execution time: **2.381 seconds** for all 133 tests in parallel.

8.2 Test Architecture

```
jest.config.js  (multi-project runner)
__tests__/
  pricing.test.ts          (16 unit tests)
  validation.test.ts       (17 unit tests)
  integration/
    materials-api.test.ts  (15 tests)
    orders-api.test.ts     (21 tests)
    gallery-api.test.ts    (9 tests)
    contact-auth-api.test.ts (8 tests)
  system/
    e2e-flows.test.ts      (24 tests)
  regression/
    regression.test.ts     (17 tests)
```

Listing 3: Jest Multi-Project Directory Structure

8.3 Unit Testing

Unit tests validate the two core business logic modules in complete isolation, with no HTTP server, no database connection, and no external dependencies.

8.3.1 Validation Module (17 tests)

Tests cover `validateOrderInput()` and `validateMaterialInput()` from `lib/validation.ts`. Key cases:

- Accepts complete valid inputs
- Rejects missing required fields (name, phone, material, dimensions)
- Rejects zero or negative numeric values (width, height, quantity, price)
- Validates email format while treating email as optional
- Collects multiple validation errors simultaneously (not just the first one)
- Trims whitespace-only strings and rejects them as blank

8.3.2 Pricing Strategy Module (16 tests)

Tests cover all three concrete strategies and the `PriceCalculator` context:

Table 16: Pricing Unit Test Cases

Strategy	Tests	Key Assertions
StandardPricing	6	Correct formula ($p \times w \times h \times q$), decimal handling, zero inputs, large orders
BulkDiscountPricing	4	No discount below qty 5; exactly 10% off at qty 5 and above
RushOrderPricing	2	Exactly 25% surcharge applied at any quantity
PriceCalculator	4	Default strategy, explicit strategy construction, runtime `setStrategy()` switching

8.4 Integration Testing

Integration tests hit live API endpoints against a running server and real PostgreSQL database. All 53 pass.

8.4.1 Materials API (15 tests)

Covers the full CRUD lifecycle: create with valid/invalid data, retrieve active/all/by-ID, update name and price, soft-delete (sets `isActive=false` rather than removing the record), and verification that soft-deleted materials are excluded from public listings but visible to admin with `?all=true`.

8.4.2 Orders API (21 tests)

The order flow is the most critical path. Tests verify:

- Server-side price recalculation (client-provided total is ignored)
- All required field validations return 400
- Foreign key validation: a nonexistent materialId returns 404
- Status pipeline enforcement: PENDING → CONFIRMED → IN_PROGRESS → COMPLETED
- Rejection of skipped transitions (PENDING → IN_PROGRESS)
- Rejection of backward transitions (CONFIRMED → PENDING)
- Terminal state enforcement (no transitions from COMPLETED)

8.4.3 Gallery API (9 tests)

Create with and without optional category field, list ordered by creation date, delete, 404 handling.

8.4.4 Contact & Auth API (8 tests)

Contact form validation (name, phone, message required; email optional). Admin login: wrong password returns 401, missing fields return 400, unauthenticated `/admin/*` requests redirect with 307.

8.5 System Testing

System tests verify complete user journeys across multiple API calls in sequence.

Table 17: System Test Flows

Flow	Description	Tests
Customer Order	Place order → retrieve by ID → verify status → confirm details	4
Admin Order Mgmt	Retrieve order → advance through full pipeline → terminal state	4
Material Management	Create → retrieve → update → soft-delete → verify exclusion	5
Gallery Management	Admin adds item → appears in public listing → admin deletes → removed	4
Page Accessibility	All 7 public pages return HTTP 200; protected routes redirect without auth	8
Total		**24**

8.6 Regression Testing

Eight previously identified bugs each have dedicated regression guards. None have reappeared.

Bug ID	Title	Fix Verified
REG-001	Uppercase enum enforcement	New orders have `PENDING` (uppercase); lowercase input rejected with 400
REG-002	Soft-delete visibility	Public GET excludes inactive; admin `?all=true` includes them
REG-003	Optional gallery category	Gallery items created with or without category; null is valid
REG-004	Status pipeline transitions	Skipped and backward transitions return 400; forward-only enforced
REG-005	Server-side price integrity	Client-provided price ignored; server recalculates from material record
REG-006	Database-connected page rendering	All public pages rendering from DB return 200 with populated content
REG-007	Duplicate material 409	Creating a material with an existing name returns 409, not 500
REG-008	Filter validation	Invalid status filter on orders returns 400, not a crash

8.7 Test Execution Output

The following screenshots were captured during a full test run on the development environment. They provide evidence of the parallel Jest multi-project execution and the individual test case results across all four test levels.

```
RUNS  integration  ...ion/gallery-api.test.ts
RUNS  unit  __tests__/validation.test.ts
    StandardPricing

RUNS  system  ...s__/system/e2e-flows.test.ts
RUNS  regression  ...ssion/regression.test.ts
RUNS  integration  ...tion/orders-api.test.ts
RUNS  integration  ...n/materials-api.test.ts
RUNS  integration  ...est.ts
RUNS  integration  ...ion/gallery-api.test.ts
RUNS  unit  __tests__/validation.test.ts
      ✓ calculates price as pricePerSqFt × width × height × quantity (8 ms)

RUNS  system  ...s__/system/e2e-flows.test.ts
RUNS  regression  ...ssion/regression.test.ts
RUNS  integration  ...tion/orders-api.test.ts
RUNS  integration  ...n/materials-api.test.ts
RUNS  integration  ...est.ts
RUNS  integration  ...ion/gallery-api.test.ts
RUNS  unit  __tests__/validation.test.ts
      ✓ scales linearly with quantity

RUNS  system  ...s__/system/e2e-flows.test.ts
RUNS  regression  ...ssion/regression.test.ts
RUNS  integration  ...tion/orders-api.test.ts
RUNS  integration  ...n/materials-api.test.ts
RUNS  integration  ...est.ts
RUNS  integration  ...ion/gallery-api.test.ts
RUNS  unit  __tests__/validation.test.ts
      ✓ handles decimal dimensions

RUNS  system  ...s__/system/e2e-flows.test.ts
RUNS  regression  ...ssion/regression.test.ts
RUNS  integration  ...tion/orders-api.test.ts
RUNS  integration  ...n/materials-api.test.ts
RUNS  integration  ...est.ts
RUNS  integration  ...ion/gallery-api.test.ts
RUNS  unit  __tests__/validation.test.ts
      ✓ returns 0 for zero dimensions

RUNS  system  ...s__/system/e2e-flows.test.ts
RUNS  regression  ...ssion/regression.test.ts
RUNS  integration  ...tion/orders-api.test.ts
RUNS  integration  ...n/materials-api.test.ts
RUNS  integration  ...est.ts
RUNS  integration  ...ion/gallery-api.test.ts
RUNS  unit  __tests__/validation.test.ts
      ✓ returns 0 for zero quantity (1 ms)
```

Figure 7: Unit Test Output (1 of 3): `StandardPricing` strategy executing within the Jest parallel runner. Test cases confirm correct area-based price calculation, linear scaling with quantity, decimal dimension handling, and zero-value edge cases.

```
      ✓ returns 0 for zero quantity (1 ms)

RUNS  system  ...s__/system/e2e-flows.test.ts
RUNS  regression  ...ssion/regression.test.ts
RUNS  integration  ...tion/orders-api.test.ts
RUNS  integration  ...n/materials-api.test.ts
RUNS  integration  ...est.ts
RUNS  integration  ...ion/gallery-api.test.ts
RUNS  unit  __tests__/validation.test.ts
      ✓ handles large orders

RUNS  system  ...s__/system/e2e-flows.test.ts
RUNS  regression  ...ssion/regression.test.ts
RUNS  integration  ...tion/orders-api.test.ts
RUNS  integration  ...n/materials-api.test.ts
RUNS  integration  ...est.ts
RUNS  integration  ...ion/gallery-api.test.ts
RUNS  unit  __tests__/validation.test.ts
    BulkDiscountPricing

RUNS  system  ...s__/system/e2e-flows.test.ts
RUNS  regression  ...ssion/regression.test.ts
RUNS  integration  ...tion/orders-api.test.ts
RUNS  integration  ...n/materials-api.test.ts
RUNS  integration  ...est.ts
RUNS  integration  ...ion/gallery-api.test.ts
RUNS  unit  __tests__/validation.test.ts
      ✓ no discount for quantity less than 5

RUNS  system  ...s__/system/e2e-flows.test.ts
RUNS  regression  ...ssion/regression.test.ts
RUNS  integration  ...tion/orders-api.test.ts
RUNS  integration  ...n/materials-api.test.ts
RUNS  integration  ...est.ts
RUNS  integration  ...ion/gallery-api.test.ts
RUNS  unit  __tests__/validation.test.ts
      ✓ applies 10% discount for quantity of 5

RUNS  system  ...s__/system/e2e-flows.test.ts
RUNS  regression  ...ssion/regression.test.ts
RUNS  integration  ...tion/orders-api.test.ts
RUNS  integration  ...n/materials-api.test.ts
RUNS  integration  ...est.ts
RUNS  integration  ...ion/gallery-api.test.ts
RUNS  unit  __tests__/validation.test.ts
      ✓ applies 10% discount for quantity greater than 5 (1 ms)

RUNS  system  ...s__/system/e2e-flows.test.ts
```

Figure 8: Unit Test Output (2 of 3): `BulkDiscountPricing` strategy. Test cases verify that no discount is applied below a quantity of five, a 10% discount is applied at exactly five units, and the discount continues to apply for quantities greater than five.

```
RUNS  system      ...s__/system/e2e-flows.test.ts
RUNS  regression  ...ssion/regression.test.ts
RUNS  integration ...tion/orders-api.test.ts
RUNS  integration ...n/materials-api.test.ts
RUNS  integration ...est.ts
RUNS  integration ...ion/gallery-api.test.ts
RUNS  unit        __tests__/validation.test.ts
      ✓ can switch strategies at runtime

RUNS  system      ...s__/system/e2e-flows.test.ts
RUNS  regression  ...ssion/regression.test.ts
RUNS  integration ...tion/orders-api.test.ts
RUNS  integration ...n/materials-api.test.ts
RUNS  integration ...est.ts
RUNS  integration ...ion/gallery-api.test.ts
RUNS  unit        __tests__/validation.test.ts
      ✓ different strategies produce different results for same input (1 ms)

RUNS  system      ...s__/system/e2e-flows.test.ts
RUNS  regression  ...ssion/regression.test.ts
RUNS  integration ...tion/orders-api.test.ts
RUNS  integration ...n/materials-api.test.ts
RUNS  integration ...est.ts
RUNS  integration ...ion/gallery-api.test.ts
RUNS  unit        __tests__/validation.test.ts


RUNS  system      ...s__/system/e2e-flows.test.ts
RUNS  regression  ...ssion/regression.test.ts
RUNS  integration ...tion/orders-api.test.ts
RUNS  integration ...n/materials-api.test.ts
RUNS  integration ...est.ts
RUNS  integration ...ion/gallery-api.test.ts
RUNS  unit        __tests__/validation.test.ts

RUNS  system      ...s__/system/e2e-flows.test.ts
RUNS  regression  ...ssion/regression.test.ts
RUNS  integration ...tion/orders-api.test.ts
RUNS  integration ...n/materials-api.test.ts
RUNS  integration ...est.ts

RUNS  system      ...s__/system/e2e-flows.test.ts
RUNS  regression  ...ssion/regression.test.ts
RUNS  integration ...tion/orders-api.test.ts
RUNS  integration ...n/materials-api.test.ts
RUNS  integration ...est.ts
```

Figure 9: Unit Test Output (3 of 3): `PriceCalculator` context class. Test cases confirm that strategies can be switched at runtime using `setStrategy()` and that different strategies applied to the same input produce different results, validating the Strategy pattern implementation.

```
RUNS  system      ...s__/system/e2e-flows.test.ts
RUNS  regression  ...ssion/regression.test.ts
RUNS  integration ...tion/orders-api.test.ts
RUNS  integration ...n/materials-api.test.ts
RUNS  integration ...est.ts

RUNS  system      ...s__/system/e2e-flows.test.ts
RUNS  regression  ...ssion/regression.test.ts
RUNS  integration ...tion/orders-api.test.ts
RUNS  integration ...n/materials-api.test.ts
RUNS  integration ...est.ts
PASS  integration __tests__/integration/contact-auth-api.test.ts

RUNS  system      ...s__/system/e2e-flows.test.ts
RUNS  regression  ...ssion/regression.test.ts
RUNS  integration ...tion/orders-api.test.ts
RUNS  integration ...n/materials-api.test.ts
RUNS  integration ...est.ts
  Contact API Integration

RUNS  system      ...s__/system/e2e-flows.test.ts
RUNS  regression  ...ssion/regression.test.ts
RUNS  integration ...tion/orders-api.test.ts
RUNS  integration ...n/materials-api.test.ts
RUNS  integration ...est.ts
    POST /api/contact

RUNS  system      ...s__/system/e2e-flows.test.ts
RUNS  regression  ...ssion/regression.test.ts
RUNS  integration ...tion/orders-api.test.ts
RUNS  integration ...n/materials-api.test.ts
RUNS  integration ...est.ts
      ✓ should accept a valid contact form submission (59 ms)

RUNS  system      ...s__/system/e2e-flows.test.ts
RUNS  regression  ...ssion/regression.test.ts
RUNS  integration ...tion/orders-api.test.ts
RUNS  integration ...n/materials-api.test.ts
RUNS  integration ...est.ts
      ✓ should accept submission without optional email (49 ms)

RUNS  system      ...s__/system/e2e-flows.test.ts
RUNS  regression  ...ssion/regression.test.ts
RUNS  integration ...tion/orders-api.test.ts
RUNS  integration ...n/materials-api.test.ts
RUNS  integration ...est.ts
      ✓ should reject submission with missing name (48 ms)
```

Figure 10: Integration Test Output: `contact-auth-api.test.ts` passing (PASS badge). The output shows individual test cases for the Contact API: valid form submission accepted (59 ms), submission without optional email field accepted (49 ms), and submission with a missing required field rejected (48 ms).

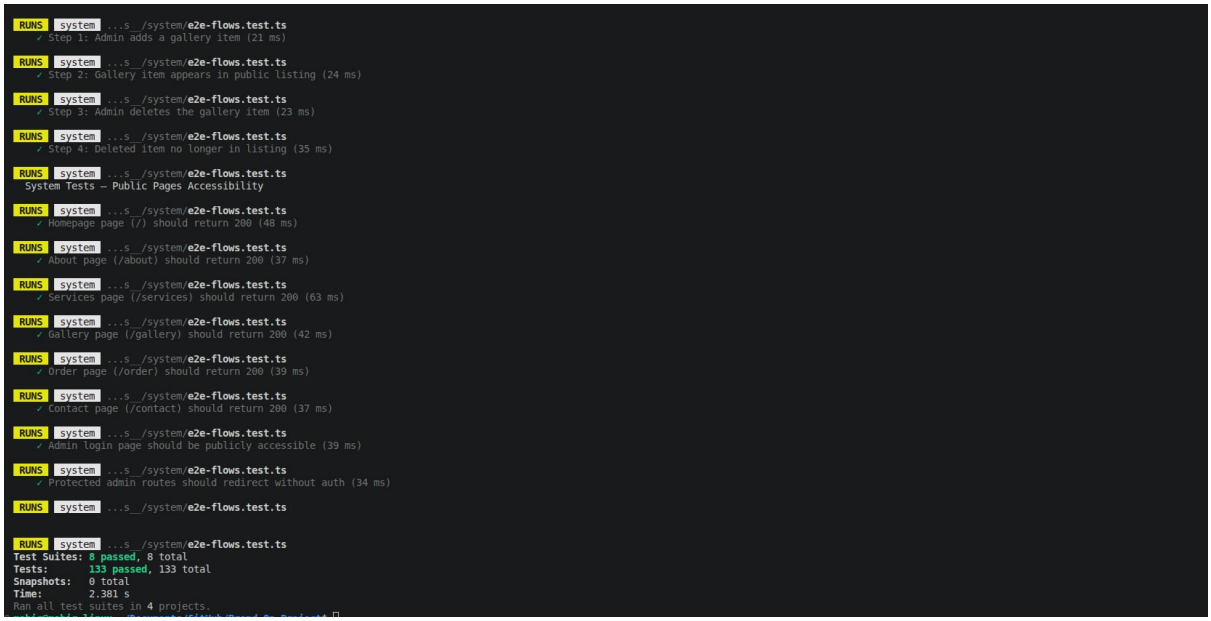

Figure 11: System Test Output and Final Summary: The upper portion shows the Public Pages Accessibility flow confirming HTTP 200 responses for all seven public routes and a redirect for protected admin routes without authentication. The summary line at the bottom shows the complete test run result: 8 suites passed, 133 tests passed, 0 snapshots, total execution time 2.381 seconds across 4 projects.

9 Conclusion

The BrandON platform delivers a complete, tested, end-to-end banner order management system built on a modern TypeScript stack. The system addresses the core problem identified during the requirements phase: customers can now browse materials, calculate prices, and place orders without visiting the shop, while the administrator has a centralised dashboard to manage all operational data.

The Singleton pattern ensured that the database connection remained stable throughout the development and testing cycles. The Strategy pattern kept the pricing logic modular and testable as new rules were introduced. The decision to calculate the total price on the server side, rather than accepting a client-submitted value, was an important security consideration that prevents price manipulation.

A total of 133 tests were executed across all four test levels and all 133 passed, with a total execution time of 2.381 seconds.

As with any software project, further improvements remain possible. These include handling additional edge cases, refining the user interface for accessibility, and extending features based on ongoing client feedback. The current implementation provides a stable foundation for these future enhancements.

Appendix

.0 A. Repository and Project Links

Resource	Link
GitHub Repository	https://github.com/M-Jawad01/Brand-On_Project
Google Drive (Deliverables)	*[Provide final Google Drive link here]*
Demo Video	*[Provide final demo video link here]*
Client Feedback Video	*[Provide client feedback recording link here]*

.0 B. API Endpoint Reference

Endpoint	Methods	Description
`/api/materials`	GET, POST	List active materials; create new material
`/api/materials/[id]`	GET, PUT, DELETE	Retrieve, update, or soft-delete a material
`/api/orders`	GET, POST	List all orders (admin); place new order (customer)
`/api/orders/[id]`	GET, PATCH	Retrieve order detail; update order status
`/api/gallery`	GET, POST	List gallery items; create gallery item
`/api/gallery/[id]`	DELETE	Delete a gallery item
`/api/upload`	POST	Upload design file, returns URL
`/api/contact`	POST	Submit contact form message
`/api/admin/login`	POST	Admin password check, sets httpOnly cookie

.0 C. Environment Variables

```
DATABASE_URL="postgresql://user:password@localhost:5432/brandon"
ADMIN_SECRET="your-admin-password-here"
```

Listing 4: Required Environment Variables (.env)

.0 D. Definition of Done

A task was considered complete only when:

1. Code compiles: `npx next build` exits with no errors
2. Feature works correctly in the browser (manual verification)
3. Relevant unit or integration tests pass
4. Code is committed and pushed to the GitHub repository
5. No TypeScript errors or ESLint warnings remain

.0 E. Test Run Commands

Command	Scope
`npx jest`	All 133 tests across all suites
`npx jest -selectProjects unit`	33 unit tests only (no server required)
`npx jest -selectProjects integration`	53 integration tests (server + DB required)
`npx jest -selectProjects system`	24 system tests
`npx jest -selectProjects regression`	17 regression tests